



TaiXin Semiconductor Linux WiFi FMAC Driver Development Guide



珠海泰芯半导体有限公司
TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港 11 栋 3 楼

Confidential Level	A	TaiXin Semiconductor Linux WiFi FMAC Driver Development Guide	Document Number	
Date	2026/7/7		Document Version	V3.9

Liability and Copyright

Limitation of Liability

THIS DOCUMENT IS INTENDED FOR REFERENCE ONLY. Zhuhai Taixin Semiconductor Co., Ltd (hereinafter referred to as "Taixin") does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information.

Taixin reserves the right to make corrections, enhancements, and other changes to this document without notice.

Taixin assumes no liability for applications assistance or the design of customers' products. **Customers are solely responsible for the design, validation, and testing of its applications as well as for compliance with all legal, regulatory, and safety-related requirements concerning its applications.**

Taixin shall not be responsible for any damages, costs, losses, and/or liabilities arising out of customer's non-compliance with this section; concurrently, Customer will fully indemnify Taixin against any damages, costs, losses, and/or liabilities arising out of customer's non-compliance with this section.

Copyright Notice

Without the the written consent of Taixin, no party shall modify, adapt, alter, translate, or create derivative works from this document for commercial purposes.

Without the written consent of Taixin, no party shall disclose or distribute any or parts of the source code, SDK, binaries and object code mentioned in this document to any third party.

No party shall modify, reverse engineer, disassemble, decompile or otherwise attempt to discover the source code of any non-source code parts of the SDK including, but not limited to pre-compiled binaries and object code.

Furthermore, any other actions that may infringe upon Taixin's rights or other intellectual property owners are strictly prohibited.

For the subject who commits the above infringement, Taixin has the right to take necessary legal measure in accordance with the laws of the People's Republic of China or other applicable laws and international treaties, including but not limited to filing a lawsuit or arbitration against the infringer, or applying for legal compulsory measures.

Zhuhai Taixin Semiconductor Co., Ltd.
September 24,2024

Confidential Level	A	TaiXin Semiconductor Linux WiFi FMAC Driver Development Guide	Document Number	
Date	2026/7/7		Document Version	V3.9

Revision History

Date	Version	Revision Notes	Reviser
2026/6/2	V3.9	Modified configuration file: connection parameters should be configured when hg0 is down. Modified psk in configuration file to wpa_psk.	WY
2026/4/29	V3.8	Modified description of MCAST_TXPARAM.	WY
2026/3/18	V3.7	Modified description of PS_MODE.	WY
2026/3/6	V3.6	Modified description of AP low-power mode.	WY
2026/2/11	V3.5	Added description of bgrssi return values. Added description of module_type.	WY
2026/1/21	V3.4	Updated parameter configuration example using hgpriv to use chan_list.	WY
2025/11/18	V3.3	Added description of disconnection reasons. Added note that dual-antenna is supported on STA only.	WY
2025/10/31	V3.2	Modified parameter configuration example file.	WY
2025/6/27	V3.1	Modified description of wakeup_io. Modified description of sysdbg. Added get mode / key_mgmt / bss_bw. Modified range of set txpower. Modified description of set tx_mcs. Added wakeup reason descriptions. Removed device model description from server keepalive command. Fixed font issues. Unified this document to support both SDK V1.x and V2.x.	WY
2025/3/24	V3.0.1	Fixed typo in wpa_psk.	WY
2025/2/26	V3.0	Added descriptions related to SDK V2.x.	WY
2024/8/12	V2.18.2	Fixed typo in super_pwr description.	WY
2024/6/20	V2.18.1	Fixed typo in get_sta_list.	WY
2024/5/5	V2. 18	Added explanation for mode3 of DCDC13.	WY
2024/4/18	V2. 17	Added interface of get signal; Modified description of low power consumption mode for AP (PS_mode4); Revised explanation of wake-up reasons for AP low power consumption;	WY
2024/2/8	V2. 16	Modified description of low power consumption mode for AP; Added description of relay function. ; Added note stating this document only supports SDK version 1. x. ;	WY
2023/12/8	V2. 15	Added explanation for superpwr=2;	WY
2023/11/29	V2. 14. 1	Adjusted descriptions for country_region/scan/paired_stas.	WY
2023/11/27	V2. 14	Added description for country_region and modified scan command.	ZEL



珠海泰芯半导体有限公司
TaiXin Semiconductor Co., Limited

3rd Floor, Gang 11 Building, 1st Jin Tang Road, Hi-tech Zone, Zhuhai China.

Copyright All Rights Reserved, Violators Will Be Prosecuted
Copyright © 2026 by TaiXin Semiconductor All rights reserved

Confidential Level	A	TaiXin Semiconductor Linux WiFi FMAC Driver Development Guide	Document Number	
Date	2026/7/7		Document Version	V3.9

2023/10/3	V2. 13	Modified explanations for wkreason and reassoc_wkhost.	WY
2023/9/20	V2. 12	Modified explanations for ps_mode and wakeup_io.	WY
2023/9/3	V2. 11	Modified description of pa_pwrctrl_dis.	WY
2023/8/31	V2. 10	Modified explanation of firmware download error messages.	DY
2023/7/28	V2. 9	Modified description of roaming.	DY
2023/5/23	V2. 8	Modified parameters explanation for multicast mode.	WY
2023/5/17	V2. 7	Modified description of roaming interface.	DY
2023/4/20	V2. 6	Added explanation for fwinfo interface. Modified description of radio_onoff interface.	DY
2023/4/13	V2. 5	Added explanation for get conn_state	DY
2023/4/7	V2. 4	Added get bssid	DY
2023/4/6	V2. 3	Modified description of roaming. Corrected typo in superpwr.	WY
2023/3/7	V2. 2. 1	Deleted parameters for several read commands.	WY
2023/2/17	V2. 2	Fixed description of sleep and wake-up. Added description of wakeup command.	WY
2023/2/6	V2. 1	Added description of dtim and autosleep. Added hyperlink for standby parameter.	WY
2023/1/12	V2. 0	Initial version.	DY

Confidential Level	A	TaiXin Semiconductor Linux WiFi FMAC Driver Development Guide	Document Number	
Date	2026/7/7		Document Version	V3.9
Table of Contents 1. Overview 1 2. Linux Kernel Compilation Configuration1 3. WiFi Driver Development User Guide 2 3.1. hgic_fmac 2 3.1.1. hgic_fmac Driver Files 2 3.1.2. hgic_fmac Loading Process2 3.1.3. hgpriv Configuration Guide 4 3.1.4. Proc fs Interfaces 21 3.1.5. Driver Event Messages22 3.1.6. One-Key Pairing24 3.1.7. STA Low Power Process 25 3.1.8. AP Low Power Mode Process28 3.1.9. Relay Function Instructions 29 3.1.10. Roaming Function Instructions (Note: Not supported by SDK V2.x standard protocol)29 3.1.11. Driver Auxiliary Module29 3.1.12. Firmware Exception Information30 3.1.13. Interface Test Mode 30 3.2. Firmware Download 31				
		珠海泰芯半导体有限公司 TaiXin Semiconductor Co. , Limited	3rd Floor, Gang 11 Building, 1st Jin Tang Road, Hi-tech Zone, Zhuhai China.	
Copyright All Rights Reserved, Violators Will Be Prosecuted Copyright © 2026 by TaiXin Semiconductor All rights reserved				

1. Overview

The TaiXin Linux WiFi FMAC driver supports running the WiFi protocol stack inside the WiFi module. The host controller does not require a WiFi stack.

The FMAC driver supports AH modules and low power modes, with the corresponding WiFi firmware being of type v2. x. x. 5.

2. Linux Kernel Compilation Configuration

The FMAC driver supports two interfaces: SDIO and USB. When compiling the kernel, the following features need to be enabled:

Depending on the interface being used (SDIO/USB), enable the corresponding support modules:

- 1) For SDIO interface: Enable "Device Driver → MMC/SD/SDIO card support" and the corresponding MMC host driver.
- 2) For USB interface: Enable "Device Driver → USB support" and the corresponding USB host driver.

Note: If you encounter an `mmc_card_disable_cd` undefined reference error during driver compilation, please enable the definition of `mmc_card_disable_cd` in `if_sdio.c` as show in the following image:

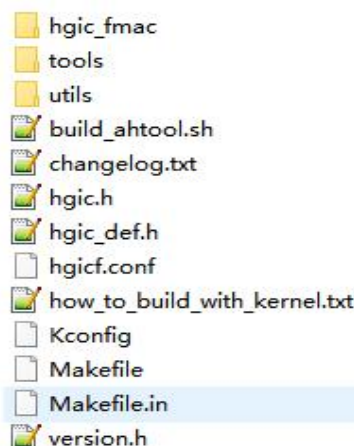
```
5:
6: #define FUNC_DEV(f)    (&((f)->dev))
7:
8: #define SDIO_CAP_IRQ(func)    ((func)->card->host->caps & MMC_CAP_SDIO_IRQ)
9: #define SDIO_CAP_POLL(func)  ((func)->card->host->caps & MMC_CAP_NEEDS_POLL)
10: #define HOST_SPI_CRC(func, crc) (func)->card->host->use_spi_crc=crc
11:
12: // #define mmc_card_disable_cd(c) (1)
13: #define hgic_card_disable_cd(func)    mmc_card_disable_cd((func)->card)
14: #define hgic_card_set_highspeed(func) mmc_card_set_highspeed((func)->card)
15: #define hgic_host_is_spi(func)        mmc_host_is_spi((func)->card->host)
16: #define hgic_card_cccr_widebus(func)  ((func)->card->cccr.low_speed && !(func)->card->cccr.high_speed)
17: #define hgic_card_cccr_highspeed(func) ((func)->card->cccr.high_speed)
18: #define hgic_host_highspeed(func)    ((func)->card->host->caps & MMC_CAP_SD_HIGHSPEED)
19: #define hgic_host_sup4bit(func)      ((func)->card->host->caps & MMC_CAP_4_BIT_DATA)
20: #define hgic_card_highspeed(func)    mmc_card_highspeed((func)->card)
21: #define hgic_func_rca(func)          ((func)->card->rca)
22: #define hgic_card_max_clock(func)   ((func)->card->cis.max_dtr)
```

3. WiFi Driver Development User Guide

3.1. hgic_fmac

3.1.1. hgic_fmac Driver Files

The hgic_fmac driver is released in source code form for users to compile on their own.



When compiling the fmac driver, you can choose to support either the sdio or usb interface. Execute the "make" command to view the compilation instructions.

```
usage:
make smac      : compile SMAC driver. support sdio/usb interface. generate hgics.ko
make smac_usb  : compile SMAC driver. only support usb interface. generate hgics.ko
make smac_sdio: compile SMAC driver. only support sdio interface. generate hgics.ko
make fmac      : compile FMAC driver. support sdio/usb interface. generate hgicf.ko
make fmac_usb  : compile FMAC driver. only support usb interface. generate hgicf.ko
make fmac_sdio: compile FMAC driver. only support sdio interface. generate hgicf.ko
make clean
```

The tools/test_app directory provides auxiliary tools and driver encapsulation APIs.

Execute build_ahtool. sh to compile the auxiliary tools, such as hgpriv and hgicf, etc.

3.1.2. hgic_fmac Loading Process

1. Loading the fmac driver: insmod hgicf. ko.

When loading the driver, the following parameters can be specified as needed:

- **ifname:** Specifies the network interface name. By default, the driver creates the hg0 interface (The latter defaults to hg0 as an example). For example: insmod hgicf. ko **ifname**="wlan%d". The driver will create the wlanx interface (x is the index obtained by %d).
- **conf_file:** Used to specify the loading parameter configuration file. The default value of this parameter is /etc/hgicf. conf. There are two ways to use it:
 - 1) The value of the conf_file parameter is a specific file path. For example:
insmod hgicf. ko conf_file=/etc/hgicf. conf. In this case, the driver will load the specified parameter file, which is suitable for single NIC solutions.
 - 2) The value of the conf_file parameter is a directory path. For example:

insmod hgicf.ko conf_file=/etc. In this case, the driver will read the /etc/hg0.conf parameter file in the /etc directory. This method is used for dual NIC solutions. For example, if two NICs, hg0 and hg1, are added in the same system, the driver will read /etc/hg0.conf and /etc/hg1.conf as parameters for hg0 and hg1, respectively.

- **fw_file:** Specifies the AH module firmware name. The driver defaults to loading hgicf.bin. See section 3.3 Firmware Download for specific instructions.
For example: insmod hgicf.ko fw_file=xxxx.bin
Note: The fw_file parameter value can only be a file name and cannot include a path.

2. Use the hgpriv tool for parameter configuration (or use the encapsulated API).

The most basic parameter settings are as follows:

- **config hg0 down**
- hgpriv hg0 set chan_list=9080,9160,9240 # Set the AH module chan_list (US standard).
- hgpriv hg0 set bss_bw=8 # Set the BSS bandwidth to 8M, which can be 1/2/4/8, consistent with the bandwidth above.
- hgpriv hg0 set tx_mcs=255 # Set the tx mcs to automatic mode.
- hgpriv hg0 set key_mgmt=NONE # Disable encryption.
- hgpriv hg0 set ssid=ah_test_ssid # Set the SSID
- hgpriv hg0 set mode=ap # Set the operating mode to ap

Parameter Configuration Sequence: Configure the channel/bandwidth information, then the encryption method/password, and finally configure the operating mode.

Note: must configure hg0 down before setting parameters for connection, otherwise connection may be fail.

3. Configure the hg0 interface: Set the IP address and bring the interface up.

4. Driver Configuration File [Optional]

If quick loading of parameters at startup is needed, a parameter configuration file can be created for the hgicf driver. Set the conf_file parameter when loading the driver, and the driver will automatically load the set parameters.

The content of the parameter configuration file consists of parameters for the hgpriv command.

For example:

- hgpriv hg0 set mode=ap, corresponds to the content in the configuration file:
mode=ap
- hgpriv hg0 set ssid=ap_ssid, corresponds to the content in the configuration file:
ssid=ap_ssid

Example file:


```

chan_list=9080,9160,9240
bss_bw=8
tx_mcs=255
acs=1, 10
key_mgmt=NONE // or WPA-PSK
wpa_psk=xxx
// 64 hex chars. The PSK must be configured after setting key_mgmt=WPA-PSK.
ssid=ah_test_ssid
// Ensure the PSK is configured before configuring the SSID.
mode=ap // or sta

```

3.1.3. hgpriv Configuration Guide

Note: It's recommended to use hgpriv tool only for manual input. For code integration, utilize the encapsulated APIs provided by the driver for enhanced convenience in application development.

- **Basic Networking Parameters**

1. **hgpriv hg0 set mode=xx**

Set the operating mode of the WiFi module: AP, STA, or Group.

- mode=ap: Operates in AP mode.
- mode=sta : Operates in STA mode.
- mode=group: Operates in Broadcast Group mode (Not supported in V2.x versions).
- mode=apsta: Operates in Repeater mode. A device in repeater mode acts both as a STA connecting to an upstream AP and as an AP providing connection services to other STAs.

Note: Set other networking parameters first before setting the mode parameter.

2. **hgpriv hg0 set ssid=xx**

Set the SSID of the AH module, up to a maximum of 32 characters.

3. **hgpriv hg0 set key_mgmt=xx**

Set the encryption mode for the WiFi module:

- NONE : Disables encryption.
- WPA-PSK: Enables encryption. (Requires setting wpa_psk)

4. **hgpriv hg0 set wpa_psk=64_hexchar**

Sets the Wi-Fi module encryption PSK. The PSK value must be 64 hexadecimal characters. This character string can be generated using the wpa_passphrase tool. For example:

```

# wpa_passphrase hgic_ah_test 12345678
network={
    ssid="hgic_ah_test"
    #psk="12345678"
    psk=baa58569a9edd7c3a55e446bc658ef76a7173d023d256786832474d737756a82
}
#

```

The red underlined section indicates the generated PSK.

Note: If you don't want to use wpa_passphrase, you can generate 64 hex characters between 0 and f yourself.

5. hgpriv hg0 set pairing=xx

Set the start/stop of the WiFi module's one-touch pairing function, which is used to control 2 WiFi modules for automatic pairing.

- pairing=1 : Starts pairing
- pairing=0 : Stops pairing

For detailed operation instructions, refer to the [one-key pairing](#) section.

6. hgpriv hg0 set bss_bw=xx

Set the BSS bandwidth at which the WiFi module operates, valid values are: 1/2/4/8

7. hgpriv hg0 set chan_list=freq0, freq1, . . . , freqN

Sets the list of operating frequencies for the WiFi module, allowing customization of the operating frequency points. Non-consecutive frequency points can be set, with units in 0.1MHz. Up to 16 frequency points are supported.

8. hgpriv hg0 set freq_range=start, end, bw

Set the operating frequency range of AH module, the unit of start and end is 0.1MHz, the unit of bw is Mhz, for example, freq_range=9080, 9160, 8:

- 9080 : Start center frequency, 908M
- 9240 : End center frequency, 924M
- 8 : bss bandwidth, 8M [This value must match bss_bw.]

Note: The channels generated by this command are continuous without gaps. If non-consecutive channels are required, you can only use the following chan_list. If chan_list is used, the freq_range setting is ignored, as chan_list takes precedence over freq_range.

9. hgpriv hg0 set country_region=xx

Set the country code for the AH module. Upon successful setting, the operating frequency points will be switched to comply with the standards of the corresponding country/region based on the bss_bw parameter. For detailed instructions, refer to the "TaiXin 802.11AH Frequency Point Configuration Guide" .

Currently, AH firmware supports frequency point settings for ten countries/regions: US (United States), EU (European Union), KR (South Korea), SG (Singapore), AU (Australia), NZ (New Zealand), ID (Indonesia), JP (Japan), MY (Malaysia), TH (Thailand), etc.

Note: This command is mutually exclusive with the set chan_list and set freq_range commands. Using the latter two commands to set frequency points will clear the country code field.

10. hgpriv hg0 set acs=1, 10

Enables the automatic frequency selection function for the WiFi module. When this function is enabled, the AH module will automatically select the frequency point with the least interference within the specified range as the operating frequency point. Typically used only in AP mode.

Parameter Description:

- 1 : Enables the function (0: disables this function).
- 10: The frequency monitoring time of the automatic frequency selection function is 10ms, and this generally does not need to be adjusted.

● **Debugging Commands**

1. hgpriv hg0 set loaddef=0/1

- loaddef=1: Restores default parameters, erasing the parameter area in the WiFi module's flash memory. After successful execution of this command, the WiFi module will restart.
- loaddef=0: No reboot after executing restore parameters

2. hgpriv hg0 set dbginfo=0/1

Enables or disables the output of firmware debugging information. When this function is enabled, debugging information from the firmware will be output to the WiFi driver and printed. This function is mainly used for capturing debugging information for problem analysis.

- hgpriv hg0 set dbginfo=1 #Enable firmware debugging information output
- hgpriv hg0 set dbginfo=0 #Disable firmware debugging information output

3. hgpriv hg0 set sysdbg=type, 0/1/2

Enables or disables debugging information output for certain categories of firmware. Type represents the categories of debugging information as follows:

- heap: Heap information Default = 0 (disabled); 1 (enabled).
- top: CPU utilization per thread. Default = 0 (disabled); 1 (enabled).
- Wnb: WiFi protocol stack information(only for SDK V1.x). Default = 0 (disabled); 1 (enabled).
- Imac: LMAC layer information. Default = 2 (concise info); 0 (disabled); 1 (complete info).
- umac: UMAC layer information (only for SDK V2.x). Default = 0 (disabled); 1 (enabled).

4. hgpriv hg0 set atcmd=at+xx

Sends AT commands to the module through the host controller.

5. hgpriv hg0 scan=0/1/2

Executes this command in STA mode to scan for surrounding AP information.

- scan=0: Stops scanning.
- scan=1: Starts scanning (does not clear previously scanned AP list before startup, caches for 10 seconds).
- scan=2: Starts scanning (clears previously scanned AP list before startup).

- **Status Query Commands**

1. **hgpriv hg0 get conn_state**

Queries the connection state of the WiFi module. Returns: 0 or 9.

```
enum hgicf_hw_state {
    HGICF_HW_DISCONNECTED = 0,
    HGICF_HW_DISABLED     = 1,
    HGICF_HW_INACTIVE     = 2,
    HGICF_HW_SCANNING     = 3,
    HGICF_HW_AUTHENTICATING = 4,
    HGICF_HW_ASSOCIATING  = 5,
    HGICF_HW_ASSOCIATED   = 6,
    HGICF_HW_4WAY_HANDSHAKE = 7,
    HGICF_HW_GROUP_HANDSHAKE = 8,
    HGICF_HW_CONNECTED    = 9,
};
```



2. **hgpriv hg0 get module_type**

Get information about the AH module type. Applications adaptively set relevant parameters based on the module type.

utils/ah_freqinfo.c defines the frequency information/transmit power for each region, and the application can integrate this part of the code to adaptively set the frequency/transmit power parameters according to different regions.

The command returns a 16-bit number, including two fields: Type (low 8 bits) and Saw (high 8 bits).

```
struct hgic_module_hwinfo{
    union{
        struct{
            unsigned char type;
            unsigned char saw:1, rev:7;
        };
        unsigned short v;
    };
};
```

Type meanings:

- 1: 700M module
- 2: 900M module
- 3: 860M module

Saw meanings:

- 0: Without SAW
- 1: With SAW

Module scan results are as follows:

- 900PNR: 258
- 900P: 2
- 900PNR-860M: 259

3. hgpriv hg0 get mode

Refer to "[set mode](#)" for the definition of mode.

4. hgpriv hg0 get ssid

Gets the SSID.

5. hgpriv hg0 get key_mgmt

Gets the encryption method; please refer to set key_mgmt.

6. hgpriv hg0 get bss_bw

Gets the bss_bw.

7. hgpriv hg0 get center_freq

Returns the center frequency of the currently used channel, in units of 100 kHz.

8. hgpriv hg0 get sta_list

View the information of currently connected STAs: AID, MAC address, PS (STA sleep state; 1 indicates sleeping), RSSI, EVM, tx_snr (RSSI - bgRSSI calculated by the peer device and fed back over the air), rx_snr (RSSI - bgRSSI calculated by the local device).

Executing this command on the AP side retrieves information about connected STAs.

Executing this command on the STA side allows viewing information about the AP to which it is currently connected.

9. hgpriv hg0 get scan_list

After executing the scan command, retrieves the list of scanned AP list.

10. hgpriv hg0 get disassoc_reason

Get the reason for connection failure:

- 0: Connection successful
- 1: Incorrect password or SSID
- 17: AP connection STA number is full
- 18: The STA is in sleep mode, and the AP disconnects or reboots

- 93: Insufficient AP memory resources
- 255: AP not found

11. hgpriv hg0 get bgrssi=chan_index

Return the bgrssi corresponding to the specified chan_index.

chan_index: Specifies the channel, starting from 1.

The return values are in the following order: max, min, avg.

12. hgpriv hg0 get bssid (Supported in SDK V1.x Only)

In STA mode, queries the MAC address of the connected AP and returns the STA's

AID. Output format: MAC, AID For example: 00: 11: 22: 33: 44: 55, 1

13. hgpriv hg0 get signal

Gets the signal strength RSSI.

Note: This interface can only be used by STAs; if used by an AP, the RSSI obtained will be inaccurate.

14. hgpriv hg0 get fwinfo

Gets firmware information.

Note: Manually executing this command will print garbled characters. Applications should use the encapsulated API: hgic_iwpriv_get_fwinfo to retrieve firmware information.

Manually executing "cat /proc/hgicf/status" can display version information.

● **Networking Advanced Parameters**

1. hgpriv hg0 set txpower=xx

Sets the maximum transmission power (unit: dBm, step: 1 dB), with a range of 1 to 20. Values outside this range will revert to the default value of 20.

2. hgpriv hg0 set super_pwr=xx

Sets whether the AH module enables the super power feature.

Enabling this feature increases the transmission power of the AH module to a maximum of 25 dBm during long-range communication. This feature is enabled by default in normal mode and disabled by default in AH test mode.

- hgpriv hg0 set super_pwr=0 #Disable super power
- hgpriv hg0 set super_pwr=1 #Enables super power, up to 25 dBm, supporting lower rates.
- hgpriv hg0 set super_pwr=2 #Enables super power, up to 22 dBm, supporting rates greater than 25 dBm but requires slightly closer proximity.

3. hgpriv hg0 set tx_mcs=xx

Sets the mcs for TX of the AH module. Valid values are: 0/1/2/3/4/5/6/7/10/255. Setting to 0/1/2/3/4/5/6/7/10 locks the MCS to a fixed tier, where 10 is only supported at 1M bandwidth.

Setting to 255 indicates automatic adjustment; the AH module will enable the auto TX rate adaptation function.

The default value is 255, indicating automatic adjustment, which is generally kept as default.

4. hgpriv hg0 set agg_cnt=xx

Sets the maximum frame aggregation count, with a default of 16. Setting a smaller aggregation count reduces average traffic, but setting it above 16 increases traffic fluctuations. It's recommended to keep the default value.

5. hgpriv hg0 set max_txcnt=xx

Sets the maximum retransmission count of WiFi frames. "0" is invalid, and "N" indicates a maximum of N transmissions, with a default of 7 times.

6. hgpriv hg0 set ap_hide=1

Enables the AP hiding feature, only valid in AP mode. After enabling AP hiding, sta will be unable to discover the AP through scanning.

7. hgpriv hg0 set bss_max_idle=xx

Set the BSS max idle time in unit S.

The STA must send a packet to the AP within the max idle time to maintain the connection. If the AP does not receive any information from the STA for longer than the max idle time, it considers the STA disconnected.

Note: This parameter affects the sleep power consumption of STA. Smaller values increase power consumption. The default is 300 seconds, corresponding to 200uA@DTIM10.

8. hgpriv hg0 set disassoc_sta=f8: de: 09: 96: 8f: 28

Disconnects a specified STA. AP can use this command to actively disconnect STA. However, in normal mode, disconnected STA will automatically reconnect.

9. hgpriv hg0 set unpair=f8: de: 09: 96: 8f: 28

Unpairs the specified MAC from the module. This command can be executed on both the AP and STA sides.

10. hgpriv hg0 set conn_paironly=1/0

After setting conn_paironly, the AP only allows STAs in the pairing list to connect. STAs not in the pairing list cannot connect even with the correct SSID and

password. Implement APs to restrict STA connections, with effects similar to whitelisting.

- `conn_paironly=1` : Only allows STAs in the pairing list to establish connections.
- `conn_paironly=0` : Allows all STAs to initiate connections, requiring the correct SSID and password. The default value is 0.

11. `hgpriv hg0 set paired_stas=mac1, mac2, . . .`

During pairing, the module automatically generates STA information, forming an STA list. If the module has flash memory, the STA list is saved to flash and automatically loaded after a module reboot. Without flash memory, the STA list cannot be saved, and is lost after a reboot.

For APs without flash memory, "set paired_stas" can be used to reset the pairing list after power-on. When paired_stas is set, and conn_paironly=1 is also set, only the specified STAs can connect to the AP, even if other STAs have the correct SSID and password, which can be achieved by restricting the connection of STAs by the AP, and the effect is similar to whitelisting.

The maximum number of STAs that can be set with this command is limited by the firmware's maximum supported number of STAs. MAC addresses are separated by commas. For example:

- `iwpriv hg0 set paired_stas=00: 11: 22: 33: 44: 55, 00: 12: 34: 56: 78: 99`

The example setup STA list contains 2 STAs with MAC addresses 00: 11: 22: 33: 44: 55 and 00: 12: 34: 56: 78: 99.

It needs to set conn_paironly=1, then set paired_stas, and finally set ap/sta mode.

12. `hgpriv hg0 set pair_autostop=xx`

Sets whether the AH module automatically stops pairing after successful pairing.

By default, AH firmware does not automatically stop pairing, and requires manually executing `hgpriv hg0 set pairing=0` to stop pairing.

- `hgpriv hg0 set pair_autostop=1` #Enables automatic pairing stop.
- `hgpriv hg0 set pair_autostop=0` #Disables automatic pairing stop.

13. `hgpriv hg0 set acktmo=xx`

Set the ack timeout value of AH module's WiFi protocol parameters, measured in microseconds, defaulting to 0. This parameter needs to be set only when conducting communication over 1km. The calculation formula is $10 * (\text{distance in kilometers} - 1)$, for example, for a distance of 2km:

`acktmo=10` #Increase ack timeout by 10us

14. `hgpriv hg0 set channel=xx`

Set the channel index to fix a working frequency point for an AH module, starting from 1.

- **Low Power Related Parameters**

1. **hgpriv hg0 set sleep=1**

Control the WiFi module to enter sleep mode.

- hgpriv hg0 set sleep=1 //Control the WiFi module to enter sleep mode
- hgpriv hg0 set sleep=0 //Clear the sleep flag recorded by the WiFi driver. When entering sleep, the WiFi driver will record the current sleep status of WiFi, blocking all access to the WiFi module. Clear this flag to continue accessing the WiFi module.

2. **hgpriv hg0 set ps_mode=xx**

Set the sleep mode of the WiFi module:

- 0: Sleep mode not set. The host can allow the module to maintain its connection with the AP when entering sleep, but the autosleep function is disabled. If there are scenarios (such as performance testing) where the device should not enter sleep automatically, set ps_mode to 0.
- 1: Module enters sleep and maintains keepalive with the server (by the module itself). This mode involves more packet transmissions and has higher sleep power consumption than Mode 2. Custom development is required.
- 2: Module enters sleep and maintains keepalive with the server (by the AP on behalf of the module). Sleep power consumption is lower than Mode 1. Only AH modules support this mode, and only UDP protocol is supported. Refer to Section 3.1.7 for the detailed keepalive procedure.
- 3: Module enters sleep and maintains keepalive with the AP only. Any unicast packet can wake up the module. Sleep power consumption is the same as Mode 2.
- 4: Module enters sleep and maintains keepalive with the AP only. Unlike Mode 3, Mode 4 cannot be woken up by any unicast packet; it can only be woken up via the AP interface /proc/hgic/wakeup. Sleep power consumption is the same as Mode 2. When server keepalive is not required, ps_mode=4 is recommended.
- 6: Module enters sleep without maintaining keepalive with the AP. The module can only be woken up by pulling wakeup_io. Sleep power consumption is lower than Mode 4 and is the lowest sleep mode available. This mode may be considered for use during transportation.

Note: In all modes listed above, the module can be woken up by pulling wakeup_io. Except for Mode 0, autosleep is enabled in all other modes. The commonly used sleep modes are: 2 (server keepalive), 4 (AP keepalive), and 6 (no keepalive).

3. **hgpriv hg0 set ap_psmode=1**

When using AP low power consumption, both the AP and STA sides need to set this parameter to help the STA quickly reconnect to the AP when restarted.

Note: When using AP low power consumption, this parameter needs to be set to 1 on both the AP and STA sides.

4. **hgpriv hg0 set wakeup_io=1, 0**

Sets the wakeup pin and trigger edge for the host to wake up the AH-TXW8301:

- 0: Rising edge (positive pulse)
- 1: Falling edge (negative pulse)

By default, MCLR is used as the wakeup pin. Note that MCLR can only be woken up using a negative pulse with a width of 500 μ s. If the design requires a positive pulse for wakeup, or if it is difficult to control the pulse width to 500 μ s, configure an available idle GPIO as the wakeup pin. Note that GPIOs other than MCLR currently support only positive pulse wakeup and do not support negative pulse wakeup. Additionally, using IOB as the wakeup pin incurs an additional 20 μ A of sleep current compared to using MCLR or IOA.

Example:

```
hgpriv hg0 set wakeup_io=34, 0
```

Sets the wakeup GPIO to IOB2 with positive pulse wakeup.

5. **hgpriv hg0 set wkio_mode=xx**

Set the working mode of AH wakeup Host IO (IOB0). Default is pulse mode.

- 1: Level mode, maintains high level when WiFi is running normally, low level during sleep.
- 0: Pulse mode, pulls up a 2ms pulse signal when the WiFi module wakes up the host controller.

6. **hgpriv hg0 set wkhost_reason=2, 7, 8, 12, 14**

Set which wake-up reasons need to wake up the host controller. Refer to the explanation of [wake reason](#) below.

7. **hgpriv hg0 get wkreason**

Query the wake-up reasons for the WiFi module(valid under the connected state).
Meaning of return value:

STA Wake-Up Reasons:

- 1: Timer wake-up in disconnected state (generally does not require waking the host)
- 2: Unicast TIM wake-up, triggered actively by the AP or by another device via unicast (generally requires waking the host)
- 3: Broadcast TIM wake-up, triggered by broadcast data (this reason has been removed)
- 4: Key IO wake-up (default: MCLR pin) (if pulled by the host, waking the host again is not necessary)
- 5: Beacon loss wake-up (generally does not require waking the host)
- 6: AP restart detection wake-up (generally does not require waking the host)
- 7: Heartbeat timeout wake-up (used in sleep Mode 2)

- 8: Wake-up packet wake-up (used in sleep Mode 2)
- 9: MCLR IO reset wake-up (reset caused by MCLR being pulled for more than 2 ms during sleep; generally does not require waking the host)
- 10: LVD low-voltage reset (does not wake the host by default)
- 11: PIR IO wake-up (requires dedicated hardware circuitry; otherwise, no configuration needed)
- 12: AP API wake-up, triggered by wnb_wakeup_sta or hgic_iwpriv_wakeup_sta on the AP side (generally requires wake-up configuration)
- 13: AP detects STA disconnection during STA sleep (generally does not require waking the host)
- 14: Wake-up upon connecting to AP while in STANDBY state (configure only when the STANDBY feature is in use)
- 16: Low-voltage wake-up (requires customer hardware support; otherwise, no configuration needed)
- 17~19: Reserved for customer-defined extensions

AP Wake-Up Reasons:

- 20: Wake-up due to AP sleep entry failure
- 21: Wake-up due to STA disconnection
- 22: Wake-up due to received STA data
- 23: Wake-up for pairing

8. hgpriv hg0 set dcdc13=xx

Setting the AH module to use external 1. 3V DCDC supply or internal LDO supply.

- hgpriv hg0 set dcdc13=0 #Using internal LDO, no external DCDC supply is required. Normal power consumption and sleep power consumption are relatively high, suitable for scenarios where power saving is not needed. This is usually the firmware default setting. When power saving is required, configure dcdc13 to one of the following modes via the interface.
- hgpriv hg0 set dcdc13=1 #Using external DCDC (hardware circuit must have 1. 3V DCDC) to power VDD13A and VDD13D. Normal power consumption and sleep power consumption are low (about 40% lower than LDO power supply). This is the recommended default for power-saving applications.
- hgpriv hg0 set dcdc13=2 # Using external DCDC (hardware circuit must have 1. 3V DCDC) to power VDD13A and VDD13D during sleep. Low sleep power consumption (consistent with dcdc13=1). During normal operation, using internal LDO to power VDD13A and VDD13D (software increases the LDO output voltage), resulting in high normal power consumption, similar to LDO power supply. Due to the high normal power consumption, this mode is not recommended.
- hgpriv hg0 set dcdc13=3 #Using external DCDC (hardware circuit must have 1. 3V DCDC) to power VDD13D. Low sleep power consumption (consistent with dcdc13=1). In normal mode, VDD13A is powered by internal LDO, while VDD13D is still powered by external DCDC. Power consumption is lower than

mode 2, but higher than mode 1 (about 20% lower than LDO power supply). This option can ensure RF performance while saving power to some extent. If using DCDC leads to poor RF EVM, consider using this option.

This parameter must be set according to the actual hardware circuit. Otherwise, if set to a non-zero value when there is no external DCDC, the device may fail to boot. If set to 0 but there is an external DCDC, power saving may not be achieved.

9. **hgpriv hg0 set pa_pwrctl_dis=xx**

Set the module's PA power control logic switch during sleep.

- hgpriv hg0 set pa_pwrctl_dis=0(default value), Does not disable PA power control logic. PA is not powered during sleep. This needs to be used in conjunction with hardware, meaning an external circuit controlled by IOA30 is added to power off the RF during sleep.
- hgpriv hg0 set pa_pwrctl_dis=1, Disables PA power control logic. PA is powered constantly.

The actual default value for this parameter is fine.

10. **hgpriv hg0 set dtim_period=xx**

Set the DTIM period under WiFi module sleep mode, in milliseconds. WiFi will regularly wake up at the specified DTIM period to check the connection with the AP and receive AP wake-ups. DTIM is set on the AP side.

DTIM10 is set to 1000 (default value), DTIM20 is set to 2000, and so on.

This parameter affects SLEEP power consumption and wake-up time. A larger DTIM value reduces SLEEP power consumption but increases wake-up time. A smaller DTIM value increases SLEEP power consumption but decreases wake-up time.

If you want to use a value less than 1000, meaning a keep-alive period of less than 1 second, besides setting dtim_period, you also need to set beacon_int reasonably. For specifics, consult with the FAE.

11. **hgpriv hg0 set autosleep_time=xx**

Set the auto sleep time of the module, in seconds, defaulting to 10 seconds, with a maximum value of 32 seconds. Setting it to -1 turns off auto sleep. Setting it to 0 also represents the default value of 10 seconds.

Auto sleep refers to the module entering sleep automatically if no commands are received from the host within the specified time after booting.

Example:

hgpriv hg0 set autosleep_time=20
Sets the auto sleep time to 20 seconds.

12. hgpriv hg0 set wait_psmode=0/1

Set the sleep mode under non-connected status, with 0 being ps connect (default) and 1 being standby mode. This is set on the STA side.

13. hgpriv hg0 set ps_connect=60, 4

The STA's WiFi module will wake up to reconnect to the AP after disconnecting while in sleep mode. If the connection fails, the WiFi module will enter PS Connect mode, cycling through sleep/wake/reconnect phases. The intermediate sleep period helps prevent excessive power consumption due to continuous reconnection attempts.

The default PS Connect behavior in the firmware is as follows: the sleep interval is 1 minute, incrementing by 1 minute for each subsequent failure up to n times.

- 1st connection failure: sleep for 1 minute
- 2nd connection failure: sleep for 2 minutes
- 3rd connection failure: sleep for 3 minutes
- 4th connection failure: sleep for 4 minutes

After incrementing the sleep time 4 times, it cycles back to the interval of the 1st attempt and repeats this pattern.

Example:

hgpriv hg0 set ps_connect=30, 4

Sets the PS Connect sleep interval to 30 seconds, with a maximum increment of 4 times.

14. hgpriv hg0 set dis_psconnect=1

If you do not want the STA to automatically enter sleep mode when not connected to an AP, you can disable PS Connect mode. For example, in certain test scenarios, you may want the device to remain awake when not connected.

Since the current sleep STA does not report information to the host after disconnection, it is recommended not to disable PS Connect mode for low-power devices under normal circumstances. Otherwise, if disconnection occurs, the device will continuously try to reconnect to the AP, and if it fails to reconnect, it will lead to rapid battery depletion.

15. hgpriv hg0 set standby=chn_idx, wakeup_time

Sets the frequency channel (starting from 1) and wake-up interval (in milliseconds) for standby mode. This should be set on the STA.

16. hgpriv hg0 set aplost_time=xx

Set the time (in seconds) for detecting AP loss when the module sleeps to stay alive. If no beacon is received from the AP within the specified time, the AP is considered lost. The default value is 10 seconds.

17. hgpriv hg0 set wkdata_save=1

Set whether the module should save wake-up data sent by the server. After saving the wake-up data, the host can read the cached wake-up data from the module via the /proc/hgic/wkdata_buff interface after booting up. The module can cache up to 4 wake-up data packets.

18. hgpriv hg0 set heartbeat=ip_str, port, hb_interval, hb_tmo

Set the heartbeat server's IP address/port number/heartbeat interval/heartbeat timeout.

Example:

hgpriv hg0 set heartbeat=192. 168. 1. 1, 6000, 60, 300

Sets the heartbeat server's IP address to 192. 168. 1. 1, port to 6000, heartbeat interval to 60 seconds, and heartbeat timeout to 300 seconds.

This command can only be used to set parameters when ps_mode=2. When ps_mode=2, before entering low-power sleep, the heartbeat application must send at least one heartbeat packet to the server. The WiFi module will capture the content of the heartbeat packet during transmission, and after entering SLEEP mode, the WiFi module will automatically send heartbeat packets to the server for keep-alive.

When using this command on the TXW80x module, only the ip_str parameter is valid; other parameters will be ignored. For secondary development of the WiFi module, refer to the "TaiXin TXW80x Low Power Development Guide"

19. hgpriv hg0 set heartbeat_resp

Set the content of the server's heartbeat response. This is used for response matching by the WiFi module to determine if the server's response has been received, indicating whether the heartbeat has been lost.

This command can only be set using the API: hgic_iwpriv_set_heartbeat_resp_data.

This parameter is required only when ps_mode=2.

20. hgpriv hg0 set wakeup_data

Set the content of the wake-up packet sent by the server to wake up the device. This is used for wake-up packet content matching by the WiFi module to determine if it needs to wake up.

This command can only be set using the API: hgic_iwpriv_set_wakeup_data

This parameter is required only when ps_mode=2.

21. hgpriv hg0 set wkdata_mask

Set the wake-up packet content matching mask for when the server wakes up the device.

This command can only be set using the API: hgic_iwpriv_set_wkdata_mask

```
int hgic_iwpriv_set_wkdata_mask(char *ifname, int offset/*from IP hdr*/, char *mask, int mask_len)
```

22. hgpriv hg0 set wakeup=mac_addr

The host executes the wake-up command to wake up a specified MAC address of a STA.

Example:

```
hgpriv hg0 set wakeup=00: 11: 22: 33: 44: 55
```

If the STA does not exist or is not in sleep mode, the return value will be -1.

23. hgpriv hg0 set reassoc_wkhost=1

When the module detects an AP anomaly during sleep, it restarts scanning and reconnects to the AP. By default, the module does not notify the host when an AP reconnection occurs. Setting reassoc_wkhost=1 causes the module to notify the host upon AP reconnection.

● Multicast Mode Parameters (Supported in SDK V1.x only)

1. hgpriv hg0 set join_group=group_addr, aid

When the WiFi module's operating mode is set to group, this command can be used to join a specific certain network. Once joined, the WiFi module will only receive data from that multicast network. All data communication will occur using the multicast address.

If the operating mode is set to group but no multicast network is joined, all data communication will be broadcast.

Note: The JOINGROUP command can only be set after the GROUP mode has been configured.

Parameters:

- group_addr: The address of the multicast network to join.
- aid: The AID for this device in the multicast network. Valid AID values: 1~N (N being the maximum number of STAs supported by the firmware). Each device in the network should have a unique AID.
 - ◆ Setting a valid AID: The WiFi module will periodically send a heartbeat in

the multicast network to announce its presence to other WiFi modules.

◆ Setting an invalid AID: The WiFi module will not send heartbeats or notify other WiFi modules. If all devices have their AID set to 0, they will not be limited by the firmware's maximum STA count

Example:

```
hgpriv hg0 set join_group=11: 22: 33: 44: 55: 66, 3
```

```
hgpriv hg0 set mcast_txparam=dupcnt, tx_bw, tx_mcs, clearch
```

Set the TX parameters for multicast communication.

- dupcnt: Sets the number of retransmissions n for multicast data. Each multicast packet is transmitted n times. Cannot be set to 0. Setting 1 means one transmission; setting 2 means two transmissions, and so on.
- tx_bw: Sets the multicast data TX bandwidth: 1 / 2 / 4 / 8.
- tx_mcs: Sets the multicast data MCS.
- Clearch: Sets whether to clear the channel before sending multicast data. This parameter is currently not in effect; set to 0.

- **Relay Parameters (Note: Definitions differ between SDK V1.x and V2.x)**

1. **hgpriv hg0 set r_ssid=xx**

- V2.x: The ssid parameter is used to connect to the upstream AP; r_ssid is used as the SSID of the relay hotspot.
- V1.x: The ssid parameter is used as the SSID of the relay hotspot; r_ssid is used to connect to the upstream AP.

2. **hgpriv hg0 set r_psk=64_hexchar**

The key value should be 64 hexadecimal characters. The usage requirements are the same as those for the wpa_psk parameter.

- V2.x: The psk parameter is used to connect to the upstream AP; r_psk is used as the password of the relay hotspot.
- V1.x: The psk parameter is used as the password of the relay hotspot; r_psk is used to connect to the upstream AP.

- **Roaming Parameters (Note: Not supported by SDK V2.x standard protocol)**

1. **hgpriv hg0 set roaming=onoff, 0, threshold, rssi_diff, rssi_int**

Enable or disable the roaming function. This command includes 5 parameters:

- Parameter 1 onoff: Takes the value 0 (off) or 1 (on), indicating whether to enable the roaming function. It is disabled by default.
- Parameter 2: Takes the value 0. This parameter is deprecated.
- Parameter 3 threshold: Sets the signal strength threshold for roaming switching. The default is -60, meaning that the STA starts looking for a stronger AP when the AP's signal strength is below -60 dBm. This can generally be increased to -50.
- Parameter 4 rssi_diff: Roaming detects the signal difference and reaches the

difference value before the AP is considered suitable for switching. The default difference is 12db, which means that the signal strength of the newly discovered AP is 12db greater than the signal of the current AP before switching is considered necessary.

- Parameter 5 rssi_interval: The roaming RSSI monitoring period, default is 5, representing 5 beacon intervals. This value affects the speed of roaming switching.

The STA needs to set this parameter, while the AP does not.

- **Dual-Antenna Parameters (Supported on STA only)**

1. **hgpriv hg0 set ant_auto=1**

Enable dual-antenna automatic selection mode.

2. **hgpriv hg0 set ant_sel=0/1**

Manually select either antenna 0 or antenna 1.

When manually selecting, the automatic selection mode needs to be turned off.

- **Other Parameters**

1. **hgpriv hg0 set auto_chswitch=xx**

Enable or disable the module's automatic channel switching feature (enabled by default).

- auto_chswitch=1 Enable
- auto_chswitch=0 Disable

The automatic channel switching feature means that when AP detects interference on the current channel during operation, it will automatically switches to another relatively clean channel. When the AP switches channels, it notifies the STA to switch as well. However, it cannot notify STAs that are in sleep mode. When the AP switches to another channel, sleeping STAs will detect the AP timeout, restart, and scan to reconnect to the AP. Once reconnected, they will return to sleep.

2. **hgpriv hg0 set auto_save=xx**

Set whether the AH module automatically saves parameters (if the AH module is configured with nor flash).

With auto-save turned off, only hgpriv hg0 save will save parameters (the parameter values for this command are saved immediately).

- hgpriv hg0 set auto_save=0 #Disable auto-save, which needs to be used with the save command below.
- hgpriv hg0 set auto_save=1 #Enable auto-save (default value).

3. **hgpriv hg0 save**

This command has no parameters. The feature is to set the AH module save parameters(if the AH module is configured with nor flash).

AH firmware automatically saves parameters by default; it detects parameter changes and saves them to flash. If auto_save is set to 0, you need to call this save command to save parameters.

4. hgpriv hg0 set dhcpc=1

Enable the module's internal DHCP Client function. The module will obtain an IP address upon booting, allowing the host controller to use the IP address directly when it starts, saving time for the host controller to apply for an IP address.

5. hgpriv hg0 set reset_sta=mac_addr

Reset the remote AH module with the specified MAC address.

6. hgpriv hg0 set radio_onoff=x

Control the wifi radio to turn it on or off.

- hgpriv hg0 set radio_onoff=0 Turn off wifi radio.
- hgpriv hg0 set radio_onoff=1 Turn on wifi radio.
- hgpriv hg0 set radio_onoff=2 Turn on wifi radio RX, Turn off TX.

3.1.4. Proc fs Interfaces

The hgic_fmac driver provides proc fs interfaces, allowing applications to interact with the driver through proc fs.

The tools/test_app directory provides iwpriv. c, which offers a wrapped API that applications can integrate to directly interact with the driver using the API.

- **/proc/hgicf/status(Read-Only)**

Running cat /proc/hgicf/status allows you to check the driver's operational status, including firmware information and driver data cache information.

This interface returns version information as a string. The fw info field format is as follows:

fw info:2.4.1.5, svn version:35402

Meaning of each field:

- 2: Major version number
- 4: Minor version number
- 1: Patch version number
- 5: FMAC project type
- 35402: Patch code version number (incremented and unique)

- **/proc/hgicf/ota(Write-Only)**

This is the module firmware OTA interface, allowing firmware upgrades for the module. If the module does not have attached Flash and the firmware is downloaded for

operation, this interface is invalid.

Usage:

- Place the firmware `firmware.bin` in the `/lib/firmware` directory.
- Execute `echo -n firmware.bin > /proc/hgicf/ot` to start the OTA upgrade process (alternatively, you can use the API: `hgic_proc_ota`). The entire process may take about 24 seconds (tested with firmware size of 320K).

If the module has attached Flash but it is empty, you can also use the OTA function to write the firmware. First, download the firmware through the interface to run it, and then use OTA to write it to Flash.

- **/proc/hgicf/fwevnt(Read-Only)**

The driver event read interface, which can receive messages from the driver and firmware by cyclically reading the interface.

Demo code is provided in the `tools/test_app/hgicf.c` file.

- **/proc/hgicf/iwpriv(Read-Only)**

This is the driver parameter setting interface. The `hgpriv.c` and `iwpriv.c` files in the `tools/test_app` directory use this interface.

The `tools/test_app/iwpriv.c` file provides a wrapped API for driver commands, allowing applications to directly invoke these commands via the API.

The `tools/test_app/hgpriv.c` file is a manual command execution tool, which can be used to manually execute driver parameter commands.

3.1.5. Driver Event Messages

The WiFi module generates various event notifications to the host during its operation. By reading the `/proc/hgicf/fwevnt` interface, you can receive event messages generated by the firmware. The `tools/test_app/hgicf.c` file provides demo code for this. For detailed parsing of each event, please refer to `hgicf.c`.

Common event descriptions:

- **HGIC_EVENT_SCANNING = 5**
Generated when the WiFi module enters scanning state.
- **HGIC_EVENT_SCAN_DONE = 6**
Generated when the WiFi module completes scanning.
- **HGIC_EVENT_TX_BITRATE = 7**
Generated every 5 seconds to report the current maximum transmission capability based on current wireless conditions. Applications can use this feedback to make adjustments, such as changing video bitrate.

-
- **HGIC_EVENT_PAIR_START = 8**
Generated when the WiFi module enters pairing mode.
 - **HGIC_EVENT_PAIR_SUCCESS = 9**
Generated when the WiFi module successfully pairs. This event continues until pairing stops and reports the MAC address of the paired STA.
 - **HGIC_EVENT_PAIR_DONE = 10**
Generated when the WiFi module stops pairing. This event reports the MAC list of all currently paired STAs.
 - **HGIC_EVENT_CONECT_START = 11**
Generated when the WiFi module starts connecting.
 - **HGIC_EVENT_CONECTED = 12**
Generated when the WiFi module successfully connects.
 - **HGIC_EVENT_DISCONNECTED = 13**
Generated when the WiFi module disconnects.
 - **HGIC_EVENT_SIGNAL = 14**
Generated when the WiFi module's wireless signal changes. This event reports the current RSSI and EVM information.
 - **HGIC_EVENT_REQUEST_PARAM = 16**
Generated when the WiFi module, acting as a STA, fails to connect to an AP. The application can reset parameters in response to this event.
 - **HGIC_EVENT_CUSTOMER_MGMT = 19,**
Generated when the WiFi module receives a custom management frame. This event reports the content of the custom management frame.
 - **HGIC_EVENT_DHCP_DONE = 21**
Generated when the WiFi module successfully executes a DHCP request to obtain an IP address.
 - **HGIC_EVENT_CONNECT_FAIL = 22**
Generated when the WiFi module fails to connect. This event reports the reason for the connection failure.
 - **HGIC_EVENT_CUST_DRIVER_DATA = 23**
Generated when the WiFi module sends custom driver data to the host.
 - **HGIC_EVENT_UNPAIR_STA = 24**

Generated when the WiFi module receives an unpairing request from the other side.

- HGIC_EVENT_EXCEPTION_INFO = 27
Generated when the WiFi module reports an abnormal condition.

3.1.6. One-Key Pairing

The TaiXin FMAC driver supports one-key pairing for networking, simplifying the configuration of AP and STA network parameters.

When initiating pairing, the AP typically needs to set parameters such as SSID/password first. If no password is set, the AP will automatically generate a random password for each STA during pairing.

During the network configuration process, the STA will receive the SSID and password information from the AP. Additionally, both the AP and STA will record each other's MAC addresses, establishing a paired STA list.

If the module has flash memory, this information will be automatically saved to the flash.

If the module lacks flash memory, this information will be lost upon reboot. In this case, the host can read and save the SSID/password/MAC address information once pairing is complete.

Notes:

1. After completing the one-key pairing, execute `set pairing=0` to stop pairing and enter the connection state.
2. One-key pairing supports pairing with only two devices simultaneously. For one-to-many network pairing, pair each STA device with the AP one by one. For example, to set up a network with one AP and four STAs, pair each of the four STA devices sequentially with the AP; do not attempt to pair all four STA devices simultaneously with the AP.
3. When the number of paired STA devices reaches the maximum, the AP will choose to overwrite the information of a disconnected STA. If no such STA is found, pairing will fail.

You can control the pairing process using the hgpriv command or the API: hgic_iwpriv_set_pairing.

- hgpriv hg0 set pairing=1 #Start pairing. Another device participating in the pairing must also enter pairing mode.
- hgpriv hg0 set pairing=0 #Stop pairing and exit pairing mode. If pairing is successful, the WiFi module will automatically establish a connection.

During the pairing process, the firmware will generate various events, which hgicf.c will read. Applications can handle these events, for example, by reading and saving SSID, password, and other information on the host side upon successful pairing.

By default, pairing does not automatically exit the pairing state after success. To configure automatic exit after successful pairing, set the pair_autostop parameter (see advanced networking parameters in hgpriv).

If pairing is unsuccessful, it will not automatically time out, so you need to manually execute hgpriv hg0 set pairing=0 to stop pairing.

3.1.7. STA Low Power Process

The TaiXin AH module supports a low-power SLEEP mode. Once in SLEEP mode, the AH module will close the interface with the host controller, resulting in access failures if the host attempts to communicate with the AH module.

There are two ways to use the SLEEP mode:

1. Keep-Alive Only with AP During SLEEP

When the SLEEP device does not need to maintain a connection with a remote server, it only needs to keep alive with the AP. In this case, the SLEEP process is simpler and follows these steps:

- 1) Enter Low Power Mode: hgpriv hg0 set sleep=1 execute the API: hgic_iwpriv_sleep
- 2) Wireless Wake-Up: Execute the API on the AP side to wake up the STA:
hgic_iwpriv_wakeup_sta
- 3) Local Wake-Up: The host controller (or a button press) can wake up the AH module by pulling the MCLR pin of the AH module low. Ensure that the wake-up pulse is not less than 100uS and not more than 1ms, with the recommended value being 500uS.
- 4) WiFi Module Wakes Up the Host: Use the IOB0 pin to wake up the host. IOB0 has two working modes, with the default being pulse mode. In this mode, IOB0 generates a pulse signal by pulling high for 2ms to wake up the host. You can change the working mode of IOB0 using the hgpriv wkio_mode command.

2. Keeping Alive with Remote Server During SLEEP

When a device in SLEEP mode needs to keep alive with a remote server, the AH module will maintain heartbeat communication with the server. Follow these steps:

- 1) Set Heartbeat Server's IP Address, Port, Heartbeat Interval, and Timeout
Configure the AH module with the IP address and port number of the heartbeat server. The AH module will automatically capture the heartbeat packets during communication and, once in SLEEP mode, will maintain the heartbeat with the server on behalf of the device. If the heartbeat times out, the WiFi module will wake up and also wake up the host via IOB0.

Use the API : hgic_iwpriv_set_heartbeat to configure these settings. Example:
`hgic_iwpriv_set_heartbeat("hg0", inet_addr("192.168.0.1"), 60002, 60, 300);`
The heartbeat server IP is 192.168.0.1, port is 60002, heartbeat interval is 60 seconds, and heartbeat timeout is 300 seconds.

2) Set Heartbeat Response Content

Configure the AH module with the content of the heartbeat response from the server. During SLEEP mode, the AH module will identify the heartbeat response based on this content. If the AH module does not receive a response after sending a heartbeat, it will attempt again on the next wake-up. After seven consecutive missed responses, the module will notify the host of a network anomaly via IO.

Use the API : `hgic_iwpriv_set_heartbeat_resp_data` to set the heartbeat response content.

3) Set Wake-Up Packet Content from Heartbeat Server

Configure the AH module with the wake-up packet content from the heartbeat server. Upon receiving and recognizing this packet, the AH module will wake up the host via IO and wake up itself.

Use the API : `hgic_iwpriv_set_wakeup_data` to set the wake-up packet content.

4) Ensure Heartbeat Communication Before Entering SLEEP

The AH module captures heartbeat packets during communication. Therefore, ensure that at least one heartbeat packet is sent before entering SLEEP so that the AH module can capture it. If the AH module does not capture a heartbeat packet, it will not maintain the heartbeat with the remote server during SLEEP.

5) Remote Wake-Up

The remote server sends a wake-up packet to the device. The WiFi module receives and recognizes this packet, waking up the device.

6) Local Wake-Up

The host controller (or a button press) can wake up the WiFi module by pulling the MCLR pin of the WiFi module low. Ensure that the wake-up pulse is not less than 100uS and not more than 1ms, with the recommended value being 500uS.

7) WiFi Module Wakes Up the Host

The WiFi module uses IOB0 to wake up the host. IOB0 is in pulse mode by default and generates a 2ms high-level pulse signal to wake up the host. You can modify the working mode of IOB0 using the `hgpriv wkio_mode` command. After the host powers on, it can query the wake-up reason via the API `hgic_iwpriv_get_wkreason`.

3. Exception Handling Process

1) AP Abnormal Reboot:

When the WiFi module in Sleep mode detects that the AP has rebooted abnormally, the WiFi module will restart and reconnect to the AP. After a successful connection, it will automatically enter sleep mode after 10 seconds. During this process, the module will not wake up the host. The module assumes the host is in sleep mode if no interaction commands are detected during this 10-second period. The

auto-sleep time can be modified using the command: iwpriv hg0 set autosleep_time

2) AP Shutdown:

When the WiFi module in sleep mode loses connection due to the AP shutting down, it will restart and attempt to reconnect to the AP. If the reconnection fails, the WiFi module will enter either PS Connect mode or standby mode, which involves cyclic sleep/wake/reconnect processes.

Select whether to enter PS Connect mode (wait_psmode=0, default) or Standby mode (wait_psmode=1) by setting [wait_psmode](#).

- PS Connect Mode: The default PS Connect behavior is as follows: the sleep interval starts at 1 minute and increases incrementally with each failure.
 - 1st failure: sleep for 1 minute
 - 2nd failure: sleep for 2 minutes
 - 3rd failure: sleep for 3 minutes
 - 4th failure: sleep for 4 minutesAfter four failures, the sleep interval cycles back to 1 minute and repeats to avoid excessive power consumption from frequent reconnections.

After a successful connection, the module will automatically enter sleep mode after 10 seconds without waking up the host.

To modify the PS Connect parameters, use: iwpriv ps_connect

To disable PS Connect mode, use: iwpriv dis_psconnect

Note: In PS Connect mode, the module restarts cyclically, leading to relatively high power consumption.

- Standby Mode: Standby mode is a new unconnected sleep mode that offers lower power consumption than PS Connect mode and allows the STA to reconnect to the AP more quickly once it powers on.

The STA needs to configure the standby parameters: the sleep channel index (starting from 1) and the wake interval (in milliseconds).

Below is the relationship between wake time and standby sleep current:

Wake Time (S)	1	3	5	10
Standby Sleep Current(uA)	500	300	250	200

The AP must specify the same sleep channel as the STA when powered on (using the [set_channel](#) command).

It is advisable to set the standby channel to the same one used by the AP before it shut down (using the [get center freq](#) command).

3) Network Disconnection:

When the WiFi module or AP in Sleep mode detects a heartbeat timeout, it indicates a network anomaly. In this situation, the WiFi module will restart and wake up the host. Once the host is powered on, it should attempt to reconnect to the server to verify network connectivity. Control is then handed over to the host, which will decide the subsequent actions.

3.1.8. AP Low Power Mode Process

The Tai Xin AH module supports AP low power mode, which can be set by the host controller. After entering low power mode, the host can choose to stand by or power down to reduce overall power consumption.

In AP low power mode, the AH module does not need to be awakened through IO and can be awakened by reinitializing the interface.

To use AP low power mode, both the AP and STA need to be configured with `hgpriv hg0 set ap_psmode=1`, and the AP must be set to `PS_MODE=4`.

1. General Process:

- Steps to enter AP low power mode:

- 1) Close Data Communication: Before entering AP low power mode, close all data communication.
- 2) Enter Low Power Mode: Execute the API: `hgic_iwpriv_sleep("hg0", 1)` to notify the AH module to enter low power mode.

- There are two ways to trigger the exit from AP low power mode:

- 1) Host Trigger: The host wakes up the module by sending a command through the interface.
- 2) STA Trigger: The STA uses the `hgic_iwpriv_wakeup_sta` API to wake up the AP. After the AP receives the wake-up packet, the AH module is not fully awake yet; it will use IOB0 to wake up the host, which then fully wakes up the module through the interface. IOB0 is in pulse mode by default, generating a 2ms high pulse signal to wake up the host. The working mode of IOB0 can be modified using the ``hgpriv wkio_mode`` command. Refer to the instructions for using this command. Note that at this point, the module is not fully awake; the host needs to initialize the SDIO interface to fully wake up the module. After the host is powered on, it can use the API ``hgic_iwpriv_get_wkreason`` to query the wake-up reason and execute the corresponding process based on different reasons. For detailed explanations of wake-up reasons, refer to the [get wkreason](#) instructions.

2. Exception Handling

- **STA loses connection after AP enters low-power mode**

In AP low-power mode, the AP can detect STA disconnection after 300 seconds.

- **AP fails to enter low-power mode**

When the AP attempts to enter low-power mode, it notifies all connected STAs. If a STA is powered off or loses signal at that moment, the AP may fail to notify that STA and therefore fail to enter low-power mode. Upon failure, the host is woken up. After wake-up, the host can query the wake-up reason and instruct the AH module to re-enter low-power mode.

- **Simultaneous AP and STA Entry into Low-Power Sleep Mode**

- SDK version 1.6: Not supported.
- SDK version 2.4: Supported.

3.1.9. Relay Function Instructions

Refer to the relevant interface settings for the relay function.

3.1.10. Roaming Function Instructions (Note: Not supported by SDK V2.x standard protocol)

The AH - STA mode supports roaming between multiple APs. The roaming function can be used as follows:

1. Enable Roaming: STA uses the command [hgpriv hg0 set roaming](#) to enable roaming. AP does not need to set roaming enable.
2. Set Roaming AP Parameters:
 - 1) Set the same SSID for each AP, or differentiate the last 3 characters of the SSID, e. g. , roaming_ssid_001, roaming_ssid_002.
 - 2) Set the same password for each AP.
3. If communication between roaming APs is required, they need to be interconnected using a wired network.

3.1.11. Driver Auxiliary Module

To simplify the application use of the fmac driver, the fmac driver package provides a driver auxiliary module: tools/test_app/iwpriv. c. This file implements encapsulated APIs for the hgpriv commands and proc interfaces, which can be directly integrated into applications. By accessing the fmac driver in the form of APIs, application development can be simplified. These encapsulated APIs parse the output information of the hgpriv/proc interfaces.

- **hgpriv Command Encapsulation API**

The hgpriv command encapsulation API is shown below. These APIs are used in the same way as the corresponding hgpriv commands, but there are two differences:

- A new **ifname** parameter is added: this parameter is the AH network interface name, such as "hg0".

- The mac parameter is a 6byte char array, while the hgpriv command uses a MAC address string.

3.1.12. Firmware Exception Information

This section introduces debugging information for exceptions in EVENTS, which can be read via /proc/hgicf/fwevnt. The hgicf.c file contains sample code:

```

break;
case HGIC_EVENT_EXCEPTION_INFO:
    exp = (struct hgic_exception_info *)data;
    switch(exp->num){
        case HGIC_EXCEPTION_TX_BLOCKED:
            printf("wireless tx blocked, maybe need reset wifi module*\r\n");
            break;
        case HGIC_EXCEPTION_TXDELAY_TOOLONG:
            printf("wireless txdelay too loog, %d:%d:%d *\r\n",
                exp->info.txdelay.max, exp->info.txdelay.min, exp->info.txdelay.avg);
            break;
        case HGIC_EXCEPTION_STRONG_BGRSSI:
            printf("detect strong backgroud noise. %d:%d:%d *\r\n",
                exp->info.bgrssi.max, exp->info.bgrssi.min, exp->info.bgrssi.avg);
            break;
        case HGIC_EXCEPTION_TEMPERATURE_OVERTOP:
            printf("chip temperature too overtop: %d *\r\n", exp->info.temperature.temp);
            break;
        case HGIC_EXCEPTION_WRONG_PASSWORD:
            printf("password maybe is wrong *\r\n");
            break;
    }
    break;

```

1. TXDELAY_TOOLONG

- Print: *wireless txdelay too long, max : min : avg *
- Description: tx delay is too long, reported when it exceeds 500ms.

2. STRONG_BGRSSI

- Print: *detect strong backgroud noise. max : min : avg *
- Description: Background noise is poor, exceeding -85dBm.

3. TEMPERATURE_OVERTOP

- Print: chip temperature too overtop
- Description: Overheating, exceeding 85 degrees Celsius.

4. WRONG_PASSWORD

- Print: password maybe is wrong
- Description: Disconnection due to encryption error on the other side, the host has an error prompt, and encryption-related parameters need to be checked.

5. TX_BLOCKED

- Print: wireless tx blocked
- Description: Transmission queue is blocked.

3.1.13. Interface Test Mode

hgic_fmac provides sdio/usbinterface test modes to perform stability tests on sdio/usbinterfaces. By specifying the `if\_test` parameter when loading the `hgicf.ko` module, you can start interface testing.

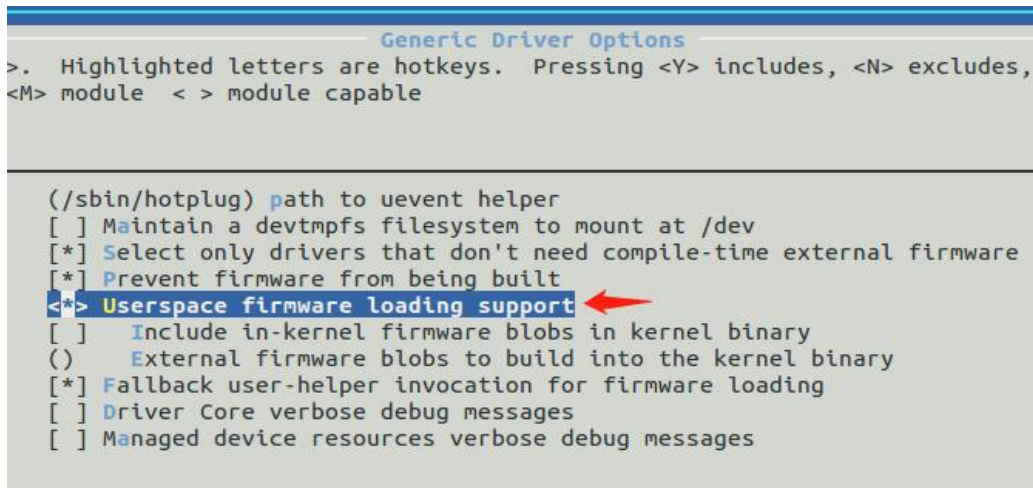
- insmod hgicf.ko if_test=1 # To start unidirectional interface testing, where data is sent from the host to the AH module
- insmod hgicf.ko if_test=2 #To start bidirectional interface testing, where data is transmitted in both directions

3.2. Firmware Download

The WiFi module supports firmware downloads via the SDIO/USB interface. The process for downloading firmware is dependent on the Linux system in use, and it may vary between different versions of the Linux kernel. The WiFi firmware should be placed in a directory supported by the system for firmware loading, typically `/lib/firmware`. If there are issues with loading the firmware, check the following aspects:

1. Kernel Compilation Configuration

Ensure that the kernel supports `CONFIG_FW_LOADER` (Devices Drivers->Generic Driver Options)



After modifying this option, the WiFi driver needs to be recompiled.

2. Check the Kernel Supported Firmware Directory

Inspect the `fw_path` array in the `firmware_class.c` file.

```
9:
9: /* direct firmware loading support */
1: static char fw_path_para[256];
2: static const char * const fw_path[] = {
3:     fw_path_para,
4:     "/lib/firmware/updates/" UTS_RELEASE,
5:     "/lib/firmware/updates",
6:     "/lib/firmware/" UTS_RELEASE,
7:     "/lib/firmware"
8: };
9:
```

Notes:

- 1) Some older kernel versions might not have the `fw_path` array in `firmware_class.c`.
- 2) The default firmware directory for Android is `/vendor/firmware` or `/etc/firmware` or `/firmware/image`

3. busybox Support for mdev

Ensure the existence of the `/sbin/mdev` file.

4. Verify Event Handler for `/proc/sys/kernel/hotplug`

Check the event handler: cat /proc/sys/kernel/hotplug. If no handler is specified, set it during system initialization: echo /sbin/mdev > /proc/sys/kernel/hotplug

5. USB Interface Sending Empty Packets

Some USB hosts do not support sending empty packets, which can cause firmware download failures. If you encounter issues similar to the ones described, verify USB Host support for empty packets or add -DCONFIG_USB_ZERO_PACKET to the Makefile.

```
07] leave hgic_bootdl_download
96] enter hgic_bootdl_download
75] hgic_get_fw_dl_addr:150::hgic_get_fw_dl_addr:check Para:download addr:20000000
18] hgic_get_fw_aes_en:114::hgic_get_fw_aes_en:check Para:aes_en:0
45] hgic_get_fw_crc_en:132::hgic_get_fw_crc_en:check Para:crc_en:1
72] hgic_get_fw_local_crc32:204::hgic_get_fw_local_crc32:check Para:local crc:0
02] hgic_get_fw_code_offset:186::hgic_get_fw_code_offset:check Para:code offset:3072
30] hgic_bootdl_parse_fw:184::firmware hdr len : 3072
51] hgic_bootdl_parse_fw:185::firmware run addr : 20000000 内核已读取固件数据
74] hgic_bootdl_parse_fw:186::firmware size : 134160
96] hgic_bootdl_parse_fw:187::firmware aes_en:0,crc_en:1
29] hgic_bootdl_send_fw:215::send fw data error, no resp!
63] hgic_bootdl_download:412::hgic_bootdl_send_fw error!
88] hgic_bootdl_download:427::Release boot download firmware... 固件下载失败
92] leave hgic_bootdl_download
25] hgic_bootdl_send_cmd_tmo:252::cmd: 0, no response!!
```

```
#FH8852
#ARCH := arm
#COMPILER := arm-fullhan-linux-uclibcgnueabi-
#LINUX_KERNEL_PATH := $(CURRENT_PATH)/../linux-3.0.8
#CFLAGS += -DFH8852 -DCONFIG_USB_ZERO_PACKET
```

6. The fw_file parameter must not contain a path.

7. Firmware Download Common Errors Instructions

- 1) Firmware Not Found: If the system reports missing firmware, verify the steps outlined above. This issue is usually due to incorrect firmware placement or the kernel not turned on to support firmware loading.
- 2) Error Code Returned by Firmware Data Download:
 - 1: Invalid command
 - 2: Invalid address (read/write error)
 - 3: Invalid length
 - 4: No permission (incorrect boot enter key)
 - 5: Command verification failure
 - 6: Data failure (CRC check error)
 - 7: Communication timeout

8. SDIO Interface Maximum Packet Length

During the firmware download phase, the WiFi driver sets the default maximum packet length to 32704 bytes. Some SD Hosts may not support such long packets. If you encounter issues, you can modify the maximum packet length in the 'if_sdio.c' file. The code needs to be changed in the following locations:

```

: sdidev->trans_cnt_addr = SDIO_TRANS_COUNT_ADDR;
: sdidev->int_status_addr = SDIO_INIT_STATUS_ADDR;
: //sdidev->status = SDIO_STATUS_STOP;
: sdidev->bus.type = HGIC_BUS_SDIO;
: sdidev->bus.drv_tx_headroom = SDIO_TX_HEADROM;
: sdidev->bus.tx_packet = hgic_sdio_tx_packet;
: sdidev->bus.tx_ctrl = hgic_sdio_tx_ctrl;
: sdidev->bus.is_busy = hgic_sdio_check_busy;
: #ifdef CONFIG_SDIO_REINIT
: sdidev->bus.reinit = hgic_sdio_reinit;
: #endif
: sdidev->bus.bootdl_pktlen = 32704;
: sdidev->bus.bootdl_cksum = HGIC_BUS_BOOTDL_CHECK_0XFD;
: sdidev->bus.probe = probe_hdl;
: sdidev->bus.dev_id = DEVID_ID(id);
: sdidev->bus.blk_size = SDIO_BLOCK_SIZE;
: sdidev->rx_retry = SDIO_CAP_IRQ(func) ? 0 : (max_pkt_len > 5 * HGIC_PKT_M
: init_completion(&sdidev->busy);

: if (!SDIO_CAP_POLL(func)) {
:     set_bit(HGIC_BUS_FLAGS_NOPOLL, &sdidev->bus.flags);
: }

48:
49: static struct hgic_bus hgic_ifbus_sdio = {
50:     .type = HGIC_BUS_SDIO,
51:     .drv_tx_headroom = SDIO_TX_HEADROM,
52:     .tx_packet = hgic_sdio_tx_packet,
53:     .reinit = hgic_sdio_reinit,
54:     .bootdl_pktlen = 32704,
55:     .bootdl_cksum = HGIC_BUS_BOOTDL_CHECK_0XFD,
56: };
57:
58: static int hgic_sdio_enable(struct sdio_func_t *func)
59: {

```

If the firmware download encounters an error as shown in the following figure: WiFi driver prompts firmware data download successfully, but cmd 4 fails. it can try to modify the sdio max packet length.

```
hgic_bootdl_download: Cmd run failed:-1
hgic_bootdl_download:425::Release boot download firmware...
leave hgic_bootdl_download
enter hgic_bootdl_download
hgic_get_fw_dl_addr:150::hgic_get_fw_dl_addr:Check Para:download addr:20001000
hgic_get_fw_aes_en:114::hgic_get_fw_aes_en:Check Para:aes_en:0
hgic_get_fw_crc_en:132::hgic_get_fw_crc_en:Check Para:crc_en:1
hgic_get_fw_local_crc32:204::hgic_get_fw_local_crc32:Check Para:local crc:0
hgic_get_fw_code_offset:186::hgic_get_fw_code_offset:Check Para:code offset:8192
hgic_bootdl_parse_fw:186::firmware hdr len : 8192
hgic_bootdl_parse_fw:187::firmware run addr : 20001000
hgic_bootdl_parse_fw:188::firmware size : 335888
hgic_bootdl_parse_fw:189::firmware aes_en:0,crc_en:1
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_bootdl_send_fw:212::Send fw data success!
hgic_fwctrl_send_data:220::timeout, ctrl->rxq:0
hgic_bootdl_send_cmd_tmo:255::cmd: 4, no response!!
hgic_bootdl_download: Cmd run failed:-1
hgic_bootdl_download:425::Release boot download firmware...
leave hgic_bootdl_download
enter hgic_bootdl_download
hgic_get_fw_dl_addr:150::hgic_get_fw_dl_addr:Check Para:download addr:20001000
```

The reason for this problem may be related to the maximum block count supported by the sd host, the default block size set by the WiFi driver is 64 bytes, and the maximum packet length is 512 blocks, so increasing the block size should also be able to solve this problem.